

PegaFlow: Predicting When Shared KV Transport Beats Recompute on Heterogeneous Accelerators

Advisor-authored storyline draft; implementation and experiments remain student-owned

M0 paper-asset closure, August 2026

Abstract

External KV-cache storage can replace repeated prefill with storage lookup and device transfer, but a cache hit is not itself a latency win. PegaFlow already provides a sidecar storage engine, a vLLM connector, host and SSD backing, and cross-node transport. Its repository also preserves both favorable and unfavorable serving traces. Those traces motivate a narrower systems question: can a phase-aware break-even contract predict when saved prefill exceeds packing, transfer, queueing, and materialization cost without reversing time-to-first-token? We formulate an architecture-guided mechanism hypothesis, a fail-closed evidence ledger, and a matched evaluation contract. Existing artifacts remain exploratory because their historical code and runtime custody is incomplete; consequently this draft reports no publishable performance result. On Ascend, the current contribution is a porting/adaptation carrier plus a falsifiable native-mechanism hypothesis, not a completed architecture-native mechanism.

1 Introduction

Long-context serving repeatedly materializes large KV states. PegaFlow moves that state outside an inference process and lets instances reuse it through a shared store. This separation creates a clean systems opportunity—replace prefill with lookup and transfer—but it also adds a new critical path. A hit must be packed or registered, moved through host and device memory, scheduled against concurrent transfers, materialized into the consumer layout, and matched to the correct request lifecycle. Average hit rate or aggregate throughput therefore cannot determine whether reuse improves a request’s SLO.

The repository’s own evidence sharpens the problem. Its preregistration states the basic inequality “saved prefill exceeds DMA cost.” Historical traces also retain regimes where burst concurrency reverses TTFT and where compressed KV leaves little useful gap. At the same time, repository issue 18 records a custody defect: most historical PegaFlow and external runtime commits are not reachable from durable refs. We use these observations only to motivate a question and freeze an evaluation; we do not promote their numbers into paper claims.

This draft makes three bounded contributions. First, it converts PegaFlow’s cache-hit story into a request-level break-even problem that accounts for the entire transfer critical path. Second, it states a phase-aware mechanism hypothesis whose output is falsifiable by matched lifecycle receipts. Third, it defines the Ascend causal chain and the custody gate required before any performance result is paper-admissible. Mechanism implementation and all new experiments remain the responsibility of the repository’s student owners.

2 The Seven-Question Research Contract

Q	Frozen answer
1	Problem. When a request can reuse shared KV state, which workload, phase, and resource conditions predict that transfer will beat isolated recompute without reversing TTFT? The unit is a matched request lifecycle, not an aggregate cache-hit ratio.
2	Importance. A wrong decision adds packing, DMA, queueing, and materialization to a request that recompute could have served sooner. The failure is user-visible tail latency and resource contention, even when average throughput or utilization improves.

- 3 **Hidden assumption and related-work boundary.** Storage caches, generic KV connectors, and transfer engines commonly treat availability or a local transfer estimate as sufficient. PegaFlow asks whether that assumption fails once request-level queue state, KV layout, accelerator copy semantics, and prefill opportunity cost interact. It is not a new cache policy, a general RPC layer, or a claim that all external-KV systems lack observability.
 - 4 **Mechanism hypothesis.** A lifecycle-bound phase model estimates $G = C_{\text{prefill}} - (C_{\text{pack}} + C_{\text{queue}} + C_{\text{transport}} + C_{\text{materialize}})$ and selects a fixed shared path only when the lower confidence bound of G clears a preregistered safety margin; otherwise it retains the matched isolated path. This selector and its Ascend-native seam are **unverified hypothesis**.
 - 5 **Feasibility.** PegaFlow already exposes connector, prefetch, DMA, queue, and request events and separates storage from the inference process. That makes a minimal receipt-bound predictor implementable at an existing seam, but feasibility is not evidence of accuracy or overhead.
 - 6 **Matched evaluation.** Compare the native isolated path, the existing fixed PegaFlow path, and the proposed phase-aware treatment using identical code/runtime pins, model and input order, service lifecycle, device allocation, KV layout, concurrency, and graph mode. Correctness must pass before TTFT, TPOT, throughput, queue time, transfer time, and utilization are analyzed.
 - 7 **Takeaway.** A reusable result would be a predictive boundary for when external KV transport is beneficial, together with a causal explanation for latency-throughput reversal. A negative result closes only the tested phase model and seam.
-

3 Problem Model and Hidden Assumption

For request r , let P_r be the prefill work avoided by a reusable prefix and let X_r be the extra work introduced by external reuse. A deployable choice needs more than $P_r > X_r$ in expectation: it must preserve output identity, respect the request and KV generation lifecycle, and meet a tail-SLO margin under contention. We decompose

$$X_r = X_r^{\text{pack}} + X_r^{\text{queue}} + X_r^{\text{transport}} + X_r^{\text{materialize}} + X_r^{\text{coordination}}.$$

The common hidden assumption is that a hit and a transfer-bandwidth estimate adequately approximate X_r . This drops shared queue delay, topology, block layout conversion, graph-mode effects, and interference among copies. It also mixes producer warmup with consumer evidence unless the lifecycle is explicit.

PegaFlow’s scope is deliberately narrower than routing or admission control. The proposed decision changes only whether a reusable KV object follows the already available shared path or the matched isolated path. It does not choose which request to serve, where to route it, how to compress its state, or which tier should own it.

4 Architecture Causal Chain for Ascend

Architecture fact. The repository’s Ascend carrier imports NPU memory across processes, uses explicit host-pinned allocation and asynchronous device copies, and records a NUMA-sensitive transfer path. The serving carrier uses graph-mode execution and a device-specific KV layout. These are code and design facts; they do not alone establish a performance contribution.

Invalidated assumption. A GPU-derived rule that treats a cache hit or nominal PCIe bandwidth as a request-level win ignores explicit import/materialization, queue contention, layout-dependent bytes, and graph-mode prefill cost. The repository’s preserved negative examples make that assumption unsafe, but their exact numeric effects remain exploratory until custody closes.

Real counterexample. Tracked serving-path traces include a burst regime in which the shared path is worse and a compressed-KV regime near break-even. They are real-path artifacts, not simulation, yet issue 18 shows that most referenced code/runtime commits are not durably retrievable. We therefore use only the qualitative contradiction: “hit implies win” is false in at least one recorded real path. No number from those traces is a paper result in this draft.

Native mechanism. The candidate native mechanism combines lifecycle receipts with an Ascend-aware phase estimate at the existing connector/transfer seam. It must account for copy-stream queue state, NUMA placement, materialized layout, and graph-mode prefill opportunity cost. No such reviewed mechanism and matched result

currently exist. Accordingly, the present Ascend work is classified as **porting/adaptation plus a pending native hypothesis**.

Predictive boundary. The shared path should win only when reusable prefill is large enough and copy contention is low enough that the lower confidence bound of G_r exceeds the safety margin for the target SLO. Short or heavily compressed prefixes, bursty concurrent copies, topology mismatch, and receiver queueing predict the opposite outcome.

Generalization class. The boundary may generalize to heterogeneous accelerators where external state reuse substitutes device compute with host or fabric movement and explicit materialization. It does not automatically generalize to in-device prefix caches or systems without lifecycle-visible transfer costs.

5 Mechanism Hypothesis

The proposed treatment has three conceptual pieces. A *receipt layer* binds lookup, enqueue, transfer, materialization, and completion to one request, KV identity, and generation. A *phase estimator* derives the expected saved prefill and added critical-path components from matched, causally valid observations. A *guarded decision* enables the shared path only when its confidence-aware gain clears the frozen margin and correctness is already qualified.

This is intentionally not an implementation specification. Student owners must choose the minimal carrier seam and demonstrate that instrumentation does not dominate the estimated gain. If the dominant delay is network or service queueing outside PegaFlow’s seam, issue 4 requires transferring the question to the appropriate networking or scheduling project instead of expanding this repository.

6 Current Evidence Ledger

Table 2 separates execution class from paper admissibility. “Real path” never overrides missing provenance.

Artifact family	Evidence label	Paper boundary
Trace-audit family	real-online trace, exploratory	Contains request and transfer events, including negative examples. Historical code/runtime custody is incomplete; no numeric claim is admitted.
Performance T1–T5 family	real-online trace audit, exploratory	Some audit manifests declare valid event conservation, while other runs fail closed. Issue 18 blocks formal use until referenced PegaFlow, vLLM, and Ascend commits and raw-log custody are durable.
Archived benchmark report	derived narrative	Historical summary only; it is not raw evidence and cannot support a result.
Trace preregistration	contract	Defines matched arms, oracles, and stop rules; it is not an experiment result.
Host and simulation tests	probe/simulation	Qualify local semantics only; they cannot establish serving, NPU, or end-to-end effects.
Future matched matrix	unverified hypothesis	Requires durable carrier/runtime pins, raw receipts, allocation identity, oracle closure, and an exact result manifest.

Table 2: Fail-closed evidence classification.

7 Evaluation Contract

7.1 Arms and matching

The strongest deployable baseline is the native isolated namespace under the same runtime and graph mode. The second baseline is fixed PegaFlow sharing with no state-aware selection. The treatment is the proposed phase-aware guarded path. Each comparison freezes the PegaFlow and runtime heads, official carrier identity, dependency lock, model and configuration hash, prompt order, request arrival trace, producer/consumer lifecycle, block layout, service flags, device allocation, clock source, and fault/interference schedule. Arm order alternates and the unit of replication is an independent service lifecycle.

7.2 Correctness oracle

Every consumer request must match the isolated baseline’s token IDs under the repository’s accepted tolerance. Receipt conservation must report exactly one eligible connector event, prefetch decision, transfer completion when required, and materialization completion per request/KV generation. Missing, duplicate, orphan, cross-arm, cross-request, or fallback-only transfer evidence invalidates the lifecycle. Producer warmup is excluded from the consumer comparison.

7.3 Metrics and preregistered gates

Report full distributions of TTFT, TPOT, total latency, request throughput, queue time, transfer time and bytes, cache-hit coverage, and accelerator/host utilization. The treatment may advance only if correctness is 100%, every claim-bearing lifecycle has complete receipts, and it beats both deployable baselines without a preregistered unacceptable TTFT or TPOT regression. Exact numeric performance thresholds remain **unverified hypothesis** until owners freeze the target SLO before observing the new matrix; this draft does not invent them.

7.4 Stop conditions

Stop after one bounded correctness-debug cycle if conservation or output identity still fails. Stop the selector hypothesis if its lower-bound decision does not repeatedly separate winning and losing regimes, if oracle headroom over the strongest fixed arm is absent, or if receipt overhead consumes the predicted gain. Stop the Ascend-native claim if only API adaptation, packaging, or environment wiring differs from the generic path. Negative outcomes close only the tested model and seam; thresholds must not be moved after held-out results are visible.

8 Related-Work Boundary

PagedAttention establishes block-granular KV-cache management inside an LLM serving engine [1], while Mooncake makes KV state a disaggregated serving resource and reports the storage-computation tradeoff of moving it across a KV-centric data path [2]. These systems motivate, but do not themselves establish, PegaFlow’s proposed receipt-bound break-even predictor. Prefix caches establish reuse within an engine; generic transfer engines optimize bytes, registration, and fabric movement; queueing and SLO controllers choose request-level service actions. PegaFlow’s proposed paper boundary is the intersection: predicting a request-level reuse break-even from lifecycle-bound transfer phases in an existing sidecar data path. A later related-work pass must add only verified references and must narrow the novelty claim if any prior system already makes the same receipt-bound prediction.

9 Limitations and Two-Week Deliverable

No result in this draft is paper-admissible. Historical runtime refs are incomplete, existing artifacts span multiple experimental revisions, and the native Ascend mechanism is unimplemented. The first week should close a claim-bearing artifact inventory and durable code/runtime refs, then freeze the minimal seam, receipts, baselines, and thresholds. The second week should produce host-level oracle tests and, only after independent allocation, one matched real-path pilot. The student owners perform all implementation, environment setup, execution, debugging, and result generation.

10 Conclusion

PegaFlow already demonstrates a substantial external-KV system carrier, but its research claim must be narrower than “cache sharing is faster.” The falsifiable opportunity is to predict the request-level break-even between saved prefill and the full transfer critical path. Until provenance custody, the native mechanism, and matched evaluation close, this remains a well-specified hypothesis with exploratory evidence rather than a completed systems result.

References

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023.

- [2] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, and Weimin Zheng. Mooncake: Trading more storage for less computation—a kvcache-centric architecture for serving llm chatbot. In *23rd USENIX Conference on File and Storage Technologies*, pages 155–170, 2025.